INSTITUTO FEDERAL DO TOCANTINS - CAMPUS COLINAS DO TOCANTINS
TECNOLOGO EM ANÁLISE E DESENVOLVIMENTO DE SISTEMAS
DANILO COSTA MATIAS

# DESENVOLVIMENTO DE UM SIMULADOR DE XADREZ COM INTEGRAÇÃO DO MOTOR STOCKFISH UTILIZANDO GODOT

COLINAS DO TOCANTINS - TO
2026

DANILO COSTA MATIAS

# DESENVOLVIMENTO DE UM SIMULADOR DE XADREZ COM INTEGRAÇÃO DO MOTOR STOCKFISH UTILIZANDO GODOT

Trabalho de curso apresentado ao curso de Tecnologo em Análise e Desenvolvimento de Sistemas do Instituto Federal do Tocantins – Campus Colinas do Tocantins, como requisito parcial para a obtenção do título de bacharel em Tecnologo em Análise e Desenvolvimento de Sistemas, sob orientação do Prof. Me. Fernando Turibio de Moura.

COLINAS DO TOCANTINS - TO

2026

**DANILO COSTA MATIAS**

# DESENVOLVIMENTO DE UM SIMULADOR DE XADREZ COM INTEGRAÇÃO DO MOTOR STOCKFISH UTILIZANDO GODOT

Trabalho de curso apresentado ao curso de Tecnologo em Análise e Desenvolvimento de Sistemas do Instituto Federal do Tocantins – Campus Colinas do Tocantins, como requisito parcial para a obtenção do título de bacharel em Tecnologo em Análise e Desenvolvimento de Sistemas, sob orientação do Prof. Me. Fernando Turibio de Moura.

**Banca Examinadora**

---

Me. Fernando Turibio de Moura
Instituto Federal do Tocantins Campus Colinas do Tocantins

---

TITULO. NOME MEMBRO 1
INTITUIÇÃO DO MEMBRO 1

---

TITULO. NOME MEMBRO 2
INSTITUIÇÃO DO MEMBRO 2

Aprovada em DATA DA DEFESA

Dedico este trabalho à minha esposa, pelo apoio incondicional, e a todos que acreditaram no meu potencial mesmo nos momentos mais desafiadores desta jornada.

## AGRADECIMENTOS

Primeiramente, a Deus, pela força, resiliência e clareza mental concedidas ao longo de toda esta jornada, permitindo-me superar os momentos de cansaço e os desafios inerentes à pesquisa e ao desenvolvimento de software.

Ao Instituto Federal de Educação, Ciência e Tecnologia do Tocantins (IFTO) - Campus Colinas do Tocantins, por ser o alicerce da minha formação profissional. Aos excelentes professores do curso de Tecnologia em Análise e Desenvolvimento de Sistemas (TADS), que não apenas compartilharam seus conhecimentos teóricos e práticos, mas também me inspiraram a buscar sempre a excelência e a inovação tecnológica.

Um agradecimento muito especial ao meu orientador, Prof. Me. Fernando Turibio de Moura. Sua paciência, dedicação e vasto conhecimento foram fundamentais para transformar uma ideia ambiciosa em um projeto real. Obrigado pela confiança no meu potencial, pelas revisões minuciosas e por me guiar com maestria pelas melhores práticas de engenharia de software, especialmente nos momentos mais complexos de integração deste simulador.

À minha esposa e à minha família, pela compreensão inesgotável nas inúmeras noites, madrugadas e finais de semana dedicados à programação, à correção de bugs e à escrita deste documento. Sem o suporte emocional, o incentivo constante e a paciência de vocês com as minhas ausências, a conclusão desta etapa não seria possível. Vocês são a minha maior motivação.

À gigantesca comunidade de desenvolvedores de software livre, em especial aos criadores e mantenedores da Godot Engine, da linguagem Python e do motor de xadrez Stockfish. A filosofia de código aberto, a documentação acessível e o compartilhamento global de conhecimento foram a base técnica que viabilizou a materialização e o sucesso deste sistema.

Aos meus colegas de turma e amigos de curso, por todas as linhas de código revisadas em conjunto, pelos testes incansáveis do jogo em suas diversas versões, pelas partidas jogadas e pelas horas de estudo compartilhadas. Vocês tornaram os desafios desta graduação muito mais leves e a jornada muito mais inesquecível.

Por fim, a todos que, de forma direta ou indireta, contribuíram com palavras de incentivo, críticas construtivas e apoio ao longo destes anos de graduação, meu mais sincero muito obrigado.

Quero que os jogadores experimentem coisas que só são possíveis dentro dos videogames e que nunca foram feitas antes. Do contrário, não há sentido em criar o jogo.

Hideo Kojima

# RESUMO

Este trabalho apresenta o desenvolvimento de um simulador de xadrez com integração ao motor de inteligência artificial Stockfish, utilizando a Godot Engine. O objetivo principal consistiu em construir uma aplicação funcional, customizável e com alto nível competitivo, solucionando o desafio técnico de integrar uma inteligência artificial complexa a um motor de jogos moderno. A metodologia adotada envolveu a criação da interface gráfica e da lógica de validação de regras, como o roque e o *en passant*, utilizando a linguagem nativa GDScript. Para o processamento das jogadas do oponente virtual, utilizou-se a linguagem Python como *middleware*, responsável por estabelecer a comunicação assíncrona entre o frontend na Godot e o motor Stockfish, operando através do protocolo *Universal Chess Interface* (UCI). Adicionalmente, foi implementado um sistema dinâmico de temas visuais baseado em dicionários e matrizes de dados, permitindo a personalização da interface gráfica em tempo real. Os resultados obtidos demonstram que a arquitetura proposta garante fluidez na jogabilidade sem comprometer o desempenho computacional durante o cálculo das melhores jogadas. Conclui-se que a integração entre Godot, Python e Stockfish forma uma base robusta, eficiente e altamente modular para o desenvolvimento de jogos de tabuleiro digitais de alta complexidade técnica.



**Palavras-chave**: Xadrez. Godot Engine. Python. Inteligência Artificial. Stockfish.

## ABSTRACT

This work presents the development of a chess simulator integrated with the Stockfish artificial intelligence engine, using the Godot Engine. The main objective was to build a functional, customizable, and highly competitive application, solving the technical challenge of integrating a complex artificial intelligence into a modern game engine. The adopted methodology involved creating the graphical interface and the rule validation logic, such as castling and *en passant*, using the native GDScript language. To process the virtual opponent's moves, the Python language was used as *middleware*, responsible for establishing asynchronous communication between the Godot frontend and the Stockfish engine, operating through the *Universal Chess Interface* (UCI) protocol. Additionally, a dynamic visual theme system based on dictionaries and data arrays was implemented, allowing for real-time customization of the graphical interface. The results obtained demonstrate that the proposed architecture ensures fluid gameplay without compromising computational performance during the calculation of the best moves. It is concluded that the integration between Godot, Python, and Stockfish forms a robust, efficient, and highly modular foundation for the development of digital board games of high technical complexity.



**Keywords**: Chess. Godot Engine. Python. Artificial Intelligence. Stockfish.

## LISTA DE ILUSTRAÇÕES

Figura 1 – Página inicial do site do sistema correlato A. . . . . 17

# LISTA DE TABELAS

## LISTA DE ABREVIATURAS E SIGLAS

| SIGLA | SIGNIFICADO DA SIGLA |
| --- | --- |

SUMÁRIO


**1 INTRODUÇÃO** . . . . . **12**

**1.1 Objetivo Geral** . . . . . 12

**1.2 Objetivos específicos** . . . . . 13

**1.3 Justificativa** . . . . . 13

**1.4 Contribuições realizadas** . . . . . 14

**1.5 Organização do trabalho** . . . . . 14

**2 REVISÃO BIBLIOGRÁFICA** . . . . . **15**

**2.1 Fundamentação teórica** . . . . . 15

2.1.1 Unified Modeling Language (UML) . . . . . 15

2.1.2 Linguagem A . . . . . 15

2.1.3 Framework A . . . . . 15

2.1.4 Framework B . . . . . 15

**2.2 Trabalhos correlatos** . . . . . 15

2.2.1 Trabalho A . . . . . 16

2.2.2 Trabalho B . . . . . 17

2.2.3 Trabalho C . . . . . 17

**3 MATERIAIS E MÉTODOS** . . . . . **18**

**4 PROJETO** . . . . . **19**

**4.1 Arquitetura do projeto** . . . . . 19

**4.2 Protótipos** . . . . . 19

**4.3 Diagrama Entidade Relacionamento (DER)** . . . . . 19

**4.4 Diagrama de Casos de Uso** . . . . . 20

**4.5 Diagrama de Classes** . . . . . 20

**4.6 Algoritmos propostos** . . . . . 20

**5 RESULTADOS E DISCUSSÃO** . . . . . **21**

**6 CONSIDERAÇÕES FINAIS** . . . . . **22**

**6.1 Contribuições** . . . . . 22

**6.2 Trabalhos Futuros** . . . . . 22

**REFERÊNCIAS** . . . . . **23**

**APÊNDICE A TITULO** . . . . . **24**

# 1 INTRODUÇÃO

O xadrez é um dos jogos de tabuleiro mais antigos e estudados do mundo, sendo frequentemente utilizado ao longo da história como um laboratório para testar a capacidade lógica e o poder de processamento dos computadores. Nos primórdios da ciência da computação, as partidas contra a máquina eram restritas a terminais de texto escuros ou interfaces extremamente rudimentares. No entanto, com a evolução da tecnologia e a mudança nas expectativas dos jogadores, os jogos digitais passaram a exigir não apenas uma matemática impecável, mas também uma apresentação visual imersiva e responsiva.

Hoje em dia, jogar xadrez contra o computador é algo comum, mas o desenvolvimento desse tipo de software envolve um desafio de arquitetura bastante específico: unir uma interface visual atraente, rica em animações e menus, com um "cérebro"lógico capaz de calcular milhões de probabilidades matemáticas em poucos segundos.

Motores de jogos modernos, como a Godot Engine, são ferramentas excelentes e acessíveis para criar telas interativas, gerenciar efeitos sonoros e construir menus complexos. Contudo, eles não foram construídos para processar algoritmos pesados de inteligência artificial de forma nativa. Por outro lado, ferramentas consagradas de Inteligência Artificial voltadas para o xadrez, como o Stockfish, são extremamente poderosas e precisas, mas funcionam apenas nos bastidores, através de linhas de comando, sem oferecer nenhuma tela, gráfico ou botão para o jogador humano.

Diante desse cenário, este trabalho apresenta o desenvolvimento do simulador digital *Royal Legacy*. O problema de pesquisa que norteia este projeto é: como integrar de forma eficiente um motor lógico de xadrez de alta complexidade (Stockfish) a uma interface gráfica moderna (Godot Engine), garantindo que o jogo seja visualmente personalizável e que os cálculos da máquina não congelem ou travem a experiência do usuário durante as partidas?

## 1.1 Objetivo Geral

Este trabalho tem como objetivo desenvolver um simulador de xadrez digital interativo e customizável na Godot Engine, integrado ao motor de inteligência artificial Stockfish através de uma comunicação em linguagem Python, proporcionando uma experiência de jogo fluida, modular e com diferentes níveis de dificuldade.

## 1.2 Objetivos específicos

Para alcançar o objetivo geral, foram definidos os seguintes objetivos específicos:

- Implementar a interface gráfica e a lógica completa das regras do xadrez (incluindo movimentação restrita, roque e *en passant*) utilizando a linguagem nativa GDScript;
- Desenvolver um *middleware* em Python para estabelecer a comunicação assíncrona entre o frontend do jogo e o binário do Stockfish, operando através do protocolo *Universal Chess Interface* (UCI);
- Estruturar um sistema de gerenciamento de cenas e menus de opções, garantindo uma experiência de usuário (UX) intuitiva para a configuração de volume, dificuldades e modos de jogo;
- Criar um sistema dinâmico de temas visuais baseado em dicionários de dados, permitindo a alteração das cores e dos conjuntos de peças em tempo real;
- Desenvolver um gerenciador de estados para controlar turnos e condições de fim de jogo (xeque-mate e empate), validando o fluxo correto das partidas nas modalidades Jogador contra Jogador (PvP) e Jogador contra Máquina (PvE).

## 1.3 Justificativa

A relevância deste trabalho reside na demonstração prática de uma arquitetura de software altamente modular. O desenvolvimento de um algoritmo de xadrez do zero, com alta performance, diretamente em uma *Game Engine* seria inviável para o escopo do projeto, além de computacionalmente ineficiente. Ao invés de reinventar ferramentas já consolidadas pela comunidade científica e de desenvolvedores, este projeto propõe uma ponte técnica elegante entre tecnologias de naturezas distintas (uma engine gráfica, uma linguagem de script utilitária e um executável externo).

Essa abordagem arquitetural é fortemente justificável pois resolve uma lacuna comum no desenvolvimento de software de entretenimento: a dificuldade de executar processos pesados em segundo plano sem impactar a fluidez visual do sistema. Muitos desenvolvedores iniciantes enfrentam travamentos de tela ao tentar calcular lances de IA na mesma via de processamento (*thread*) dos gráficos. Ao documentar a separação dessas responsabilidades, este projeto serve

como um modelo de estudo valioso para estudantes de Análise e Desenvolvimento de Sistemas que precisem integrar motores visuais modernos a inteligências artificiais complexas.

## 1.4 Contribuições realizadas

O desenvolvimento deste simulador trouxe contribuições práticas e metodológicas para a estruturação de projetos de jogos de tabuleiro, destacando-se:

- **Arquitetura de Comunicação Assíncrona:** A implementação de um script em Python que atua como ponte, traduzindo o estado do tabuleiro (formato FEN) e devolvendo a melhor jogada (formato UCI), provou que é viável acoplar programas externos pesados à Godot sem causar interrupções (*stutters*) na interface principal.
- **Modularidade de Interface:** A centralização da navegação entre os diversos menus e telas do jogo por meio de um único gerenciador global demonstrou a aplicação de boas práticas de design de software, facilitando a expansão do código e evitando o acoplamento excessivo entre as telas.
- **Escalabilidade Visual:** A criação de um sistema de customização que utiliza estruturas de dados flexíveis (como Dicionários) substituiu lógicas condicionais repetitivas, resultando em um código mais limpo e otimizado para futuras atualizações de conteúdo estético.

## 1.5 Organização do trabalho

Este capítulo expôs o contexto inicial, o problema de pesquisa, os motivos, os objetivos e as contribuições deste trabalho. O capítulo 2 apresenta os principais conceitos teóricos sobre o xadrez computacional, assim como as ferramentas, linguagens e *frameworks* utilizados para a solução do problema. O capítulo 3 apresenta a metodologia adotada e a arquitetura delineada para o desenvolvimento da aplicação. O capítulo 4 expõe o projeto de *software* construído, detalhando os scripts, a organização visual e o fluxo de dados entre o jogo e a Inteligência Artificial. O capítulo 5 apresenta os resultados práticos obtidos com o simulador em funcionamento, discutindo a eficiência da comunicação assíncrona. Por fim, o capítulo 6 compila as considerações finais sobre os desafios superados e sugere propostas para trabalhos futuros.

## 1.6 Trabalhos correlatos

Esta seção apresenta os principais softwares, plataformas e repositórios relacionados ao tema deste estudo. A análise destes sistemas permite não apenas identificar os padrões da indústria, mas também reconhecer as bases comunitárias que apoiaram o desenvolvimento arquitetural do simulador *Royal Legacy*.

### 1.6.1 Godot With Me: Chess (Repositório Open-Source)

O projeto *Godot With Me: Chess* é um repositório de código aberto hospedado no GitHub que serve como um guia prático para a implementação das regras básicas de xadrez utilizando a Godot Engine. Ele aborda a movimentação em *grid* (grade) e o instanciamento das peças 2D no tabuleiro (**??**).

Este repositório serviu como principal referência inicial para o desenvolvimento do *Royal Legacy*, auxiliando no entendimento da manipulação de matrizes em GDScript e na estruturação visual das casas lógicas. No entanto, o presente trabalho diferencia-se e expande significativamente o escopo desse repositório base ao implementar três grandes melhorias estruturais: a integração de uma Inteligência Artificial externa (Stockfish) operando em segundo plano via Python; a criação de um gerenciador de cenas estruturado para menus complexos; e um sistema dinâmico de troca de temas visuais via Dicionários de dados.

### 1.6.2 Lichess (Plataforma Web Open-Source)

O Lichess é um dos maiores servidores de xadrez da Internet, sendo totalmente gratuito e de código aberto. A plataforma permite partidas *multiplayer* e contra a máquina, utilizando o próprio motor Stockfish nos seus servidores (*backend*) ou através da compilação em WebAssembly no navegador do usuário (*frontend*).

A figura **??** apresenta o tabuleiro padrão da plataforma. Embora seja uma referência mundial, o Lichess possui uma arquitetura focada em tecnologias web (cliente-servidor). O presente trabalho difere por ser uma aplicação *desktop* nativa focada em renderização através de uma *Game Engine*, dispensando a necessidade de navegadores para a integração com a Inteligência Artificial.

Figura 1 – Interface de partida contra o computador na plataforma Lichess.

./capitulos/revisao_bibliografica/lichess.png

### 1.6.3 Lucas Chess

O *Lucas Chess* é um software *desktop* voltado para o treinamento e ensino de xadrez. O seu grande diferencial é a inclusão de dezenas de motores de IA diferentes, configurados para simular adversários de diversas faixas de força (rating ELO), de iniciantes a Grandes Mestres.

Assim como o projeto *Royal Legacy*, o Lucas Chess utiliza a integração local com motores via protocolo UCI. No entanto, a sua interface gráfica é construída com ferramentas tradicionais de janelas do sistema operacional, focando mais na utilidade estatística do que na experiência audiovisual ou fluidez de animações. O uso da Godot Engine neste TCC propõe justamente uma evolução na apresentação visual e interativa desse tipo de simulador local.

### 1.6.4 Chess.com

O Chess.com é a plataforma comercial líder de mercado em xadrez digital. A aplicação oferece uma vasta gama de personalização de temas, avatares (bots) com personalidades distintas e análises profundas de partidas. Por ser um software proprietário, os motores lógicos

utilizados por trás dos seus "bots"são fechados em servidores remotos de alto desempenho. O desenvolvimento do *Royal Legacy* busca espelhar a experiência de customização visual e a seleção de dificuldades de IA encontrada no Chess.com, porém arquitetando tudo de forma local, *offline* e utilizando apenas ferramentas *open-source*.

# 2 MATERIAIS E MÉTODOS

Os materiais e métodos compõem a metodologia do trabalho. Neste momento deverão ser apresentadas as principais tecnologias utilizadas.

Entende-se como materiais:

- linguagens de programação (Julia, Java, PHP, Python);
- *frameworks* ou arcabouços;
- Banco de dados (PostgreSQL, MySQL, SQL Server, Oracle, etc.);
- Ambiente de Desenvolvimento Integrado (IDE): NetBeans, Eclipse, etc;
- APIs de sistemas;
- *Hardwares*: seu computador[1], Arduíno, Raspberry Pi, etc.

Não entenda mal, a questão deste capítulo não consiste apenas em citar os materiais, mas sim falar onde e como eles foram utilizados na sua pesquisa. Então ao citar o computador, deve ser mencionado nos métodos como o computador foi utilizado.

Como métodos entende-se:

- metodologia de desenvolvimento (SCRUM, cascata, incremental, etc.);
- qual forma o sistema foi modelado? Usou-se Diagrama de Fluxo de Dados (DFD)? Unified Modeling Language (UML)? O que foi usado?
- Como os requisitos foram levantados? Quais técnicas foram usadas nas entrevistas? Onde foram as entrevistas? Quando? Quem eram os envolvidos?
- Quais etapas foram seguidas no desenvolvimento?

O autor deve tomar cuidado para não confundir os materiais com a revisão bibliográfica (capítulo 2). Note que neste capítulo não é o momento de definir o que são os materiais, mas sim como eles foram utilizados. Então se o autor vai apresentar a linguagem Julia ao leitor, isto deve ser feito no capítulo de revisão bibliográfica na parte de fundamentação teórica (seção 2.1).

[1] Sim! Seu computador. Se estiver fazendo um trabalho que envolva calcular tempo de execução de um algoritmo, as definições do seu computador devem ser incluídas no trabalho, tais como: processador, número de núcleos, número de *threads*, memória RAM, tipo de disco rígido (HD ou SSD), tamanho do disco rígido, até mesmo a velocidade de conexão deve ser incluída dependendo do trabalho.

# 3 PROJETO

Este capítulo apresenta os principais elementos de projeto do sistema proposto. Neste sentido são definidas estruturas da arquitetura do sistema, protótipos e diagramas estruturais e comportamentais.

Embora este não seja o momento de definir conceitos[1], você pode endossar o porquê as coisas foram feitas do jeito que foram utilizando citações.

## 3.1 Arquitetura do projeto

Fazer um diagrama de implantação da UML ou fazer uma figura que expresse como as partes do sistema foram divididas e como elas se comunicam.

## 3.2 Protótipos

Nesta seção são apresentados os principais, senão todos, os protótipos do sistema. Conceitue o que são protótipos e lembre que eles são rascunhos do sistema, não as telas finais. As telas finais são tema do capítulo de resultados e discussão (capítulo 5).

Apresente todos os protótipos sempre com uma explicação e a figura.

## 3.3 Diagrama Entidade Relacionamento (DER)

Nesta seção encontra-se um exemplo de conceito que normalmente não entra no capítulo de revisão, mas cai muito bem uma boa citação.

Por que é importante citar aqui? Veja qual o motivo de ter escolhido um banco de dados relacional? E por que a modelagem foi feita da forma que foi? Ela segue as normas formais? Quantas normas formais são? Segue todas elas?

Então, além de apresentar a figura do DER você também deve fazer uma explicação e afirmar que ela segue as normas formais. Em casos que não seguirem as normas formais deve ser prontamente justificado o porquê.

É importantíssimo conferir a coesão do DER com os protótipos do sistema apresentados na seção 4.2. Todos os campos apresentados no protótipo devem estar no DER.

[1] O momento de definir conceitos importantes é na revisão bibliográfica, capítulo 2.

## 3.4 Diagrama de Casos de Uso

Inclua e explique o diagrama de caso de uso.

## 3.5 Diagrama de Classes

Inclua e explique o diagrama de caso de classes. Note que o diagrama de classes pode ter classes de domínio e sua relação assim como também podem haver diagramas de classes com mais níveis de detalhamento. Caso seja necessário inclua mais detalhes.

## 3.6 Algoritmos propostos

No seu trabalho você não deve incluir códigos[2]. Dê preferência a incluir os algoritmos que representam contribuições claras do seu trabalho. Veja se está fazendo um sistema de *e-commerce*, inclua o algoritmo de indicação de produtos, por exemplo o algoritmo 1. E se o sistema for de biblioteca, inclua o algoritmo de sugestão de livros. E assim vai, converse com seu orientador o que seriam os algoritmos mais relevantes do seu trabalho.

**Algoritmo 1:** Algoritmo de indicação de produtos.

```
P ← pesquisar_produtos();
n ← length(P);
se n ≥ 1 então
    P_ord ← ordenar_por_estrelas(P);
    P_preferidos ← P_ord[1 : 10];
    retorna P_preferidos
senão
    retorna Lista vazia
fim
```

[2] Raramente isto é aceitável

# 4 RESULTADOS E DISCUSSÃO

Esta seção é dinâmica e dependerá muito do seu trabalho. Mas nela podem ser incluídas as principais telas desenvolvidas no sistema. Discussões sobre diferenças entre o protótipo e a tela.

Preferencialmente peça para seu usuário julgar as telas do seu sistema. Para isto elabore uma metodologia, que deve ser incluída no capítulo 3, de avaliação do sistema desenvolvido.

Nesta metodologia não precisa ser complexa, basta colocar uma ficha contendo notas para aparência, facilidade de uso, tempo de resposta, utilidade, etc. Facilite sua análise pedindo para o usuário dar notas de 0 a 10 ou em outra escala, o importante é que seja numérica.

Debata sobre estes resultados nesta seção. Observe que se seu trabalho é desenvolver um novo algoritmo e os critérios de avaliação são claros, não é necessário que um usuário o julgue. Veja, você está desenvolvendo um novo algoritmo de ordenação, a complexidade e medições de do tempo gasto na ordenação já bastarão[1].

[1] Note que isto não evita que a metodologia de análise seja definida no capítulo 3.

# 5 CONSIDERAÇÕES FINAIS

Este capítulo apresenta as considerações finais do trabalho. Sendo dividido em duas seções 6.1 e 6.2. Na primeira, serão apresentados os avanços proporcionados por este trabalho. Na segundo, são apresentados pontos de melhorias para que futuros pesquisadores continuem contribuindo com o tema.

## 5.1 Contribuições

Apresente de forma pontual as contribuições que fez, ex:

- O algoritmo de ordenação do problema X foi melhorado para $O(n \log n)$;
- A interface do sistema ficou mais intuitiva[1];

## 5.2 Trabalhos Futuros

Aqui serão apresentadas propostas para continuação da linha de pesquisa deste tema. Sugira coisas que acredita que devem ser melhoradas em médio e longo prazo.

[1] Cuidado com estas afirmações, estabeleça uma metodologia no capítulo 3 para que estas afirmações estejam corretas.

## REFERÊNCIAS

TANENBAUM, A.; FEAMSTER, N.; WETHERALL, D. **Rede de Computadores**. 6. ed. Porto Alegre: Pearson, 2021. ISBN 978-85-8260-561-5.

WAZLAWICK, R. S. **Metodologia de Pesquisa em Ciência da Computação**. 3. ed. Rio de Janeiro: LTC, 2021. ISBN 978-85-951-5109-3.

# APÊNDICE A – TITULO